1. Operadores Relacionales

Los **operadores relacionales** en Python se utilizan para comparar dos valores y devuelven un resultado de tipo **booleano (True o False)**. Son fundamentales para estructuras de control como **condicionales (if)**

1.1. Resultado de una comparación relacional

Independientemente del tipo de datos involucrados, toda comparación relacional devuelve siempre un valor booleano:

- True si la comparación es verdadera
- False si la comparación es falsa

Estos valores booleanos serán utilizados posteriormente para tomar decisiones dentro de un programa.

Operador	Descripción	Ejemplo	Resultado
==	Igual a	5 == 5	True
!=	Distinto de	5 != 3	True
>	Mayor a	5 > 8	False
<	Menor a	3 < 1	False
>=	Mayor o igual a	5 >= 4	True
<=	Menor o igual a	4 <= 2	False

1.2. Comparaciones con variables

Además de comparar valores literales, los operadores relacionales se utilizan frecuentemente para comparar variables.

Por ejemplo:

```
edad = 17
print(edad >= 18)
```

✓ 0.0s

```
False
```

En este caso, Python evalúa la expresión edad >= 18 utilizando el valor almacenado en la variable edad y devuelve True o False según corresponda.

También es posible comparar el contenido de dos variables entre sí:

```
precio_actual = 1500
precio_maximo = 2000

print(precio_actual <= precio_maximo)
```
✓ 0.0s

```
True
```

1.3. Comparación de distintos tipos de datos

Los operadores relacionales pueden utilizarse para comparar distintos tipos de datos, siempre que la comparación tenga sentido.

1.3.1. Comparación de números

Las comparaciones numéricas siguen el orden matemático habitual:

```
print(10 > 3)
print(7 <= 5)
```
✓ 0.0s

```
True
False
```

1.3.2. Comparación de cadenas de texto (strings)

Cuando se comparan cadenas de texto en Python, la comparación no se realiza por el significado de las palabras, sino por el valor numérico de cada carácter, según su código ASCII (o Unicode).

Python compara los caracteres uno por uno, comenzando desde el primero, hasta encontrar una diferencia.

Por ejemplo:

```
print("Ana" < "Juan")
```
✓ 0.0s

```
True
```

En este caso, Python compara "A" con "J".
Como el código ASCII de "A" es menor que el de "J", la expresión devuelve True.

También es importante tener en cuenta que las mayúsculas y minúsculas tienen valores distintos, lo cual influye en el resultado de la comparación:

```
print("a" > "A")
```
✓ 0.0s

```
True
```

Esto ocurre porque las letras minúsculas tienen un código ASCII mayor que las mayúsculas.

Por este motivo, al comparar cadenas de texto, es importante ser consistente con el uso de mayúsculas y minúsculas.

1.4. Diferencia entre asignación y comparación

Es importante no confundir el operador de asignación con el operador de comparación:

- = se utiliza para asignar un valor a una variable
- == se utiliza para comparar dos valores

Por ejemplo:

```
x = 5        # asignación
print(x == 5)  # comparación
```

Confundir estos operadores es uno de los errores más comunes al comenzar a programar.